



CodeSmile

Test Incident Report

Pre – modification

RIFERIMENTO	TestIncidentReport- PreModification
VERSIONE	0.4
DESTINATARIO	Prof. Andrea De Lucia Dott. Giammaria Giordano



Laurea Magistrale in Software Engineering & IT Management –Università di Salerno
Corso di Ingegneria del Software: tecniche avanzate
Prof. Andrea De Lucia, Dott. Giammaria Giordano

Gestione Gruppo

MEMBRO	EMAIL	MATRICOLA
MARTINA DE LUCIA	m.delucia18@studenti.unisa.it	NF22500053
ALESSANDRA RAIA	a.raia7@studenti.unisa.it	NF22500054
CARLA STEFANILE	c.stefanile1@studenti.unisa.it	NF22500097



Version

DATA	VERSIONE	DESCRIZIONE	AUTORI
22/01/2026	0.1	Prima Stesura	MDL, AR, CS
22/01/2026	0.2	Report test di unità	AR
04/02/2026	0.3	Report test di sistema	CS
05/02/2026	0.4	Revisione + test di sistema	AR



Sommario

Gestione Gruppo.....	2
Version.....	3
1 Introduzione.....	5
2 Esecuzione dei Test di Unità.....	5
2.1 Dettagli del fallimento.....	5
2.2 Risultato.....	6
3 Esecuzione dei Test di Sistema.....	7
3.1 Analisi Trasversale dei fallimenti.....	7
3.2 Risultato.....	8



1 Introduzione

Il presente documento consiste in un report dell'esito dell'esecuzione delle attività di testing sul progetto **CodeSmile**, nello stato del sistema antecedente all'implementazione delle modifiche previste dalle Change Request, come definito nei documenti Test Plan Pre Modification e Test Case Specification Pre Modification.

L'obiettivo del documento è:

- documentare in modo strutturato e tracciabile le anomalie riscontrate;
- fornire evidenza oggettiva dello stato di qualità del sistema pre-modification;
- supportare la costruzione di una baseline affidabile per il confronto con i risultati del testing successivo all'introduzione delle Change Request.

2 Esecuzione dei Test di Unità

La seguente tabella esplicita i test di Unità eseguiti, lo status (cioè se i test sono passati o no) e i dettagli (numericamente rilevante per avvalorare lo status)

MODULO	STATUS	DETTAGLI
cli	PASSED	16/16 test superati
code_extractor	PASSED	34/34 test superati
components	FAILED	12/13 test superati (1 fallito)
detection_rules	PASSED	74/74 test superati
gui	PASSED	5/5 test superati
report	PASSED	7/7 test superati
TOTALE	148/149	

2.1 Dettagli del fallimento

Nome del test:	test_analyze_project_empty_directory
Modulo coinvolto:	components/project_analyzer.py
Errore riportato:	ValueError: The project 'test/unit_testing/components/mock_project_path' contains no Python files.
Causa del problema:	
Il test intende verificare il comportamento dell'applicazione quando analizza una cartella che non contiene file Python. Il fallimento è dovuto a due fattori:	
1. Mocking applicato nel punto sbagliato (root cause tecnica)	
Il test applica il monkeypatch a <code>utils.file_utils.FileUtils.get_python_files</code> , ma <code>ProjectAnalyzer</code>	



utilizza il simbolo FileUtils importato nel modulo components.project_analyzer. Di conseguenza, il mock non intercetta la chiamata effettiva a `get_python_files()` (“patch where defined” invece di “patch where used”).

2. **Aspettativa del test non coerente con il comportamento implementato (root cause funzionale)**

L’implementazione di `analyze_project()` considera la mancanza di file Python un caso d’errore e **solleva esplicitamente una ValueError**, mentre il test si aspetta un ritorno pari a 0 (graceful handling).

Conclusione: il test fallisce perché non patcha correttamente la funzione usata dal codice e, in ogni caso, non gestisce l’eccezione prevista dal comportamento attuale.

Soluzione:

Per allineare il test al comportamento attuale del componente e rendere efficace il mocking:

1. **Correzione del monkeypatch nel namespace corretto**

È stato applicato il mock a `components.project_analyzer.FileUtils.get_python_files` (ovvero nel punto in cui il simbolo è effettivamente utilizzato da ProjectAnalyzer), garantendo che `get_python_files()` ritorni una lista vuota.

2. **Aggiornamento dell’asserzione del test**

Il test è stato aggiornato per verificare il comportamento previsto dall’implementazione, cioè il sollevamento di una `ValueError` quando non sono presenti file Python, usando `pytest.raises(ValueError)` (eventualmente con `match` sul messaggio).

2.2 Risultato

MODULO	STATUS	DETTAGLI
components	PASSED	13/13 test superati
TOTALE	149/149	



3 Esecuzione dei Test di Sistema

La seguente tabella esplicita i test di Sistema eseguiti, lo status (cioè se i test sono passati o no) e i dettagli (numericamente rilevante per avvalorare lo status)

MODULO	STATUS	DETTAGLI
gui_system_spec	PASSED	17/43 test superati
TOTALE	17/43	

3.1 Analisi Trasversale dei fallimenti

L'analisi complessiva dei test di sistema evidenzia che la maggior parte dei fallimenti non è riconducibile a errori algoritmici nell'analisi dei progetti, bensì a carenze nella gestione e validazione delle configurazioni di esecuzione.

In particolare, i test dal mettono in luce un problema sistematico legato all'assenza di un controllo centralizzato di coerenza tra:

- numero di progetti selezionati;
- modalità *Multiple*;
- modalità *Parallel*;
- modalità *Resume*.

Il sistema consente infatti l'attivazione di combinazioni logicamente incoerenti (ad esempio modalità *Parallel* o *Resume* senza *Multiple*, oppure modalità *Multiple* con un solo progetto), degradando automaticamente l'esecuzione in modalità sequenziale senza fornire alcuna comunicazione esplicita all'utente.

Questa degradazione silenziosa produce due effetti principali:

1. **Mancanza di trasparenza:** l'utente non è informato del comportamento effettivamente adottato dal sistema.
2. **Assenza di validazione preventiva:** configurazioni semanticamente non valide non vengono intercettate prima dell'avvio dell'analisi.

Il problema assume quindi una natura prevalentemente architetturale e di progettazione dell'interfaccia di configurazione, piuttosto che funzionale rispetto al motore di analisi.

Dal punto di vista della qualità del software, tale comportamento incide su:

- affidabilità percepita;
- prevedibilità dell'esecuzione;
- coerenza tra configurazione selezionata e comportamento reale del sistema.



Una possibile evoluzione progettuale consiste nell'introdurre un modulo di validazione preliminare delle configurazioni, responsabile di verificare la coerenza tra parametri e modalità prima dell'avvio dell'analisi, bloccando l'esecuzione o producendo messaggi informativi espliciti in caso di configurazioni non supportate.

3.2 Risultato

I test, a seguito delle modifiche relative alla Change Request CR00, raggiungono i seguenti risultati, accuratamente analizzati nel documento ReportDiModifica:

MODULO	STATUS	DETTAGLI
gui_system_spec	PASSED	43/43 test superati
TOTALE	43/43	